Binary Tree Traversal Techniques

- Three recursive techniques for binary tree traversal
- In each technique, the left subtree is traversed recursively, the right subtree is traversed recursively, and the root is visited
- What distinguishes the techniques from one another is the order of those 3 tasks

Preoder, Inorder, Postorder

- In Preorder, the root is visited before (pre) the subtrees traversals
- In Inorder, the root is visited in-between left and right subtree traversal
- In postorder, the root is visited after (post) the subtrees traversals

Preorder Traversal:
1. Visit the root
2. Traverse left subtree
3. Traverse right subtree

Inorder Traversal:
1. Traverse left subtree
2. Visit the root
3. Traverse right subtree

Postorder Traversal:
1. Traverse left subtree
2. Traverse right subtree
3. Visit the root

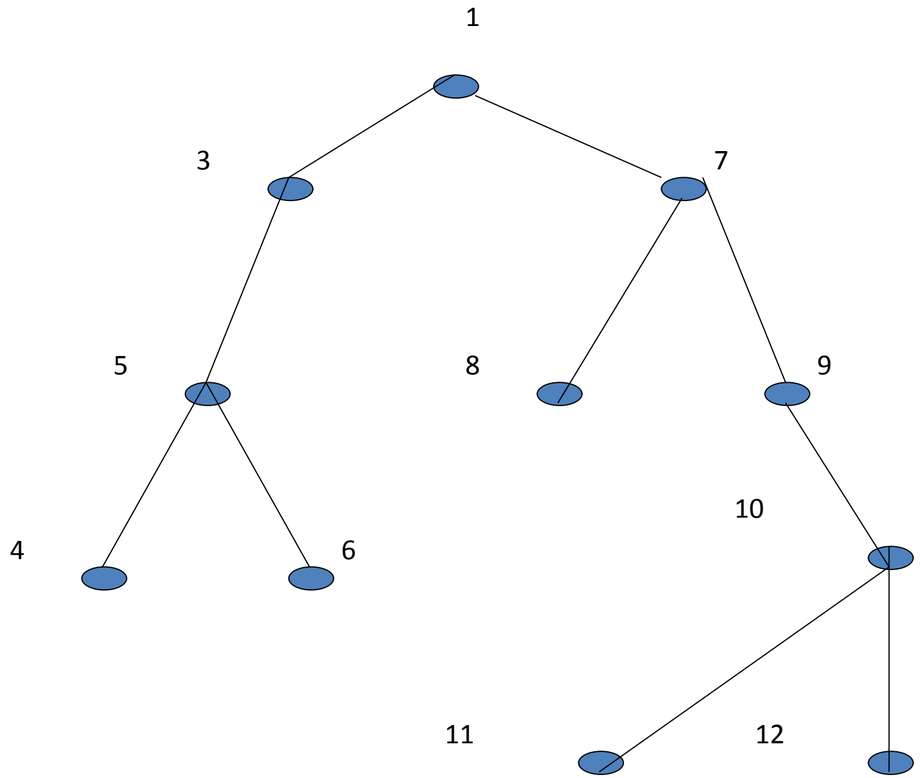
Illustrations for Traversals

- Assume: visiting a node is printing its label
- Preorder:

  1 3 5 4 6 7 8 9 10 11 12
- Inorder:

  4 5 6 3 1 8 7 9 11 10 12
- Postorder:

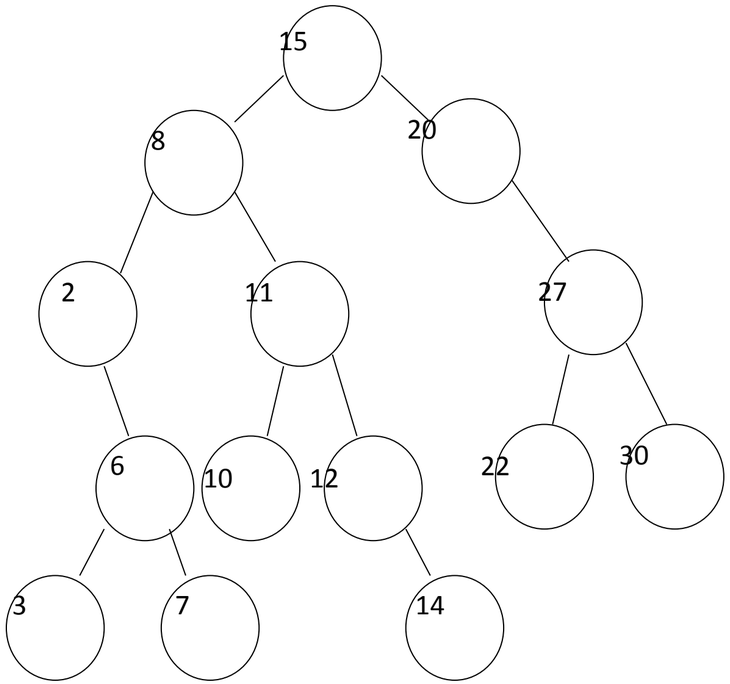
  4 6 5 3 8 11 12 10 9 7 1



Illustrations for Traversals (Contd.)

- Assume: visiting a node is printing its data

- Preorder: 15 8 2 6 3 7 11 10 12 14 20 27 22 30
- Inorder: 2 3 6 7 8 10 11 12 14 15 20 22 27 30
- Postorder: 3 7 6 2 10 14 12 11 8 22 30 27 20 15



Binary Search Tree

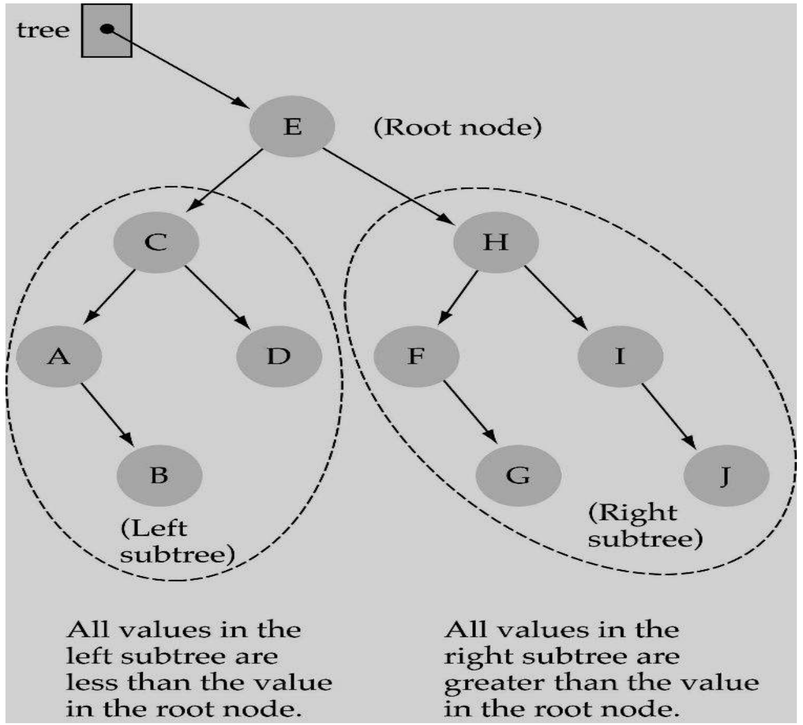
- **Binary Search Tree** is a node-based binary tree data structure which has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the node’s key.
- The right subtree of a node contains only nodes with keys greater than the node’s key.
- The left and right subtree each must also be a binary search tree.

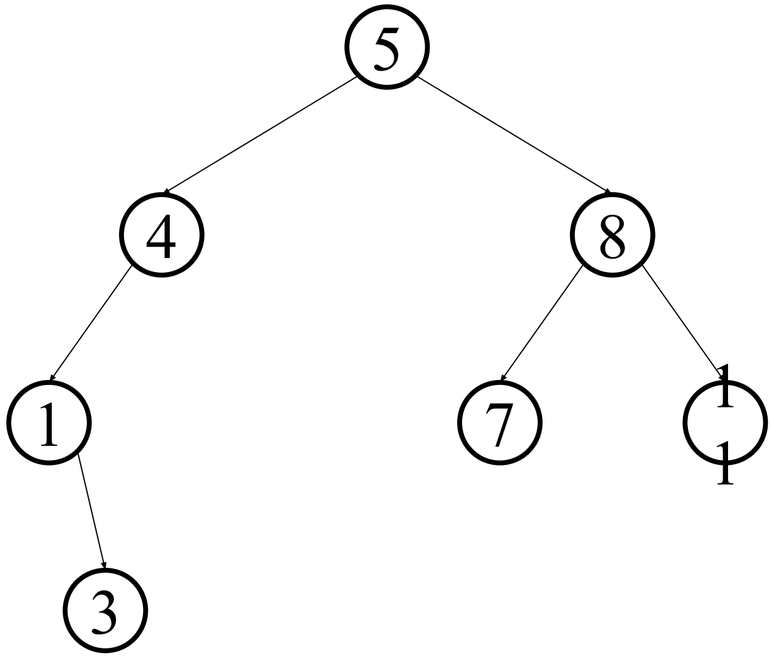
  There must be no duplicate nodes.

Binary **Search** Trees (BSTs)

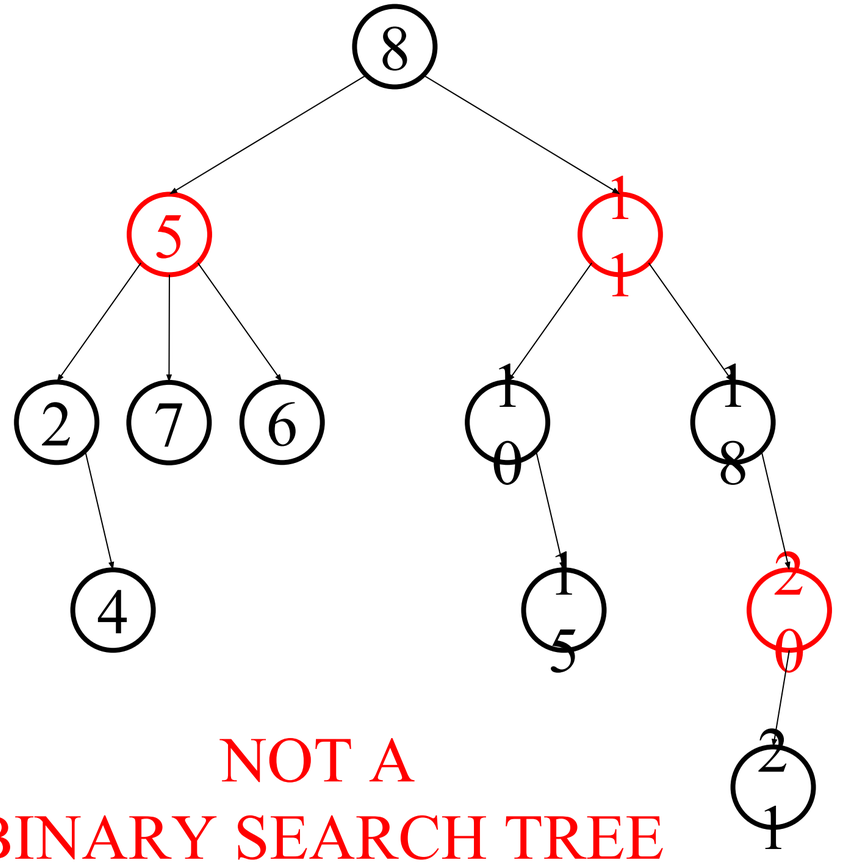
- **Binary Search Tree Property**:
  The value stored at a node is *greater* than the value stored at its left child and *less* than the value stored at its right child



Binary Search Tree Example



BINARY SEARCH TREE



NOT A
BINARY SEARCH TREE

Properties of Binary Search Tree

- Properties of Binary Search Tree provides an ordering among keys so that the operations like search, minimum and maximum can be done fast.

- If there is no ordering, then we may have to compare every key to search for a given key.

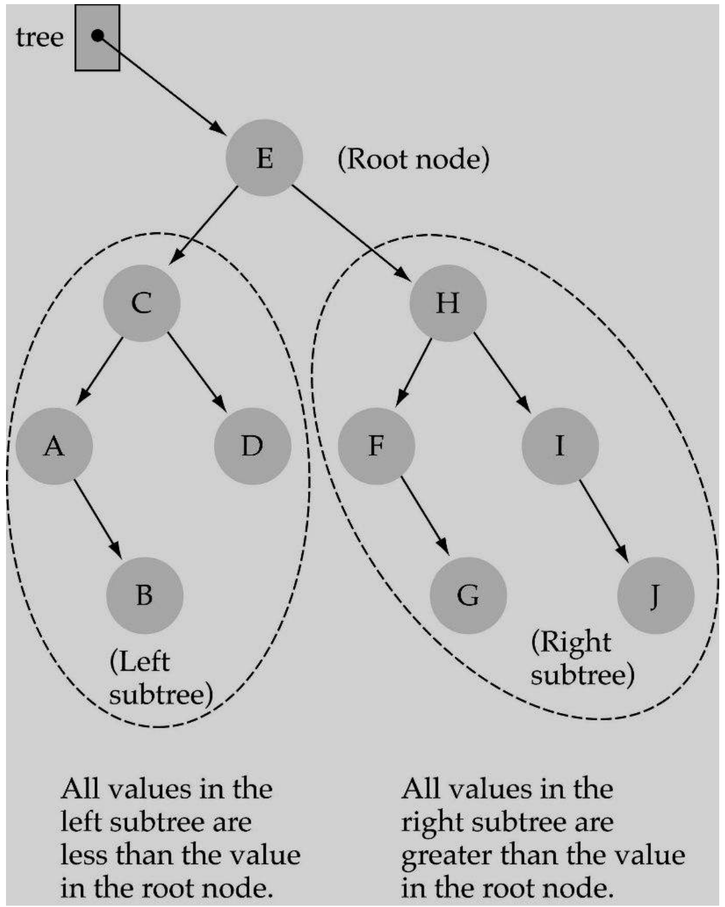
Binary **Search** Trees (BSTs)

Where is the smallest element?

Ans: leftmost element

Where is the largest element?

Ans: rightmost element



Creation of BST from the given list

- 76,32,7,8,23,56,67,12,35,20,59,21

Basic Operations

Following are the basic operations of a tree −

Search − Searches an element in a tree.

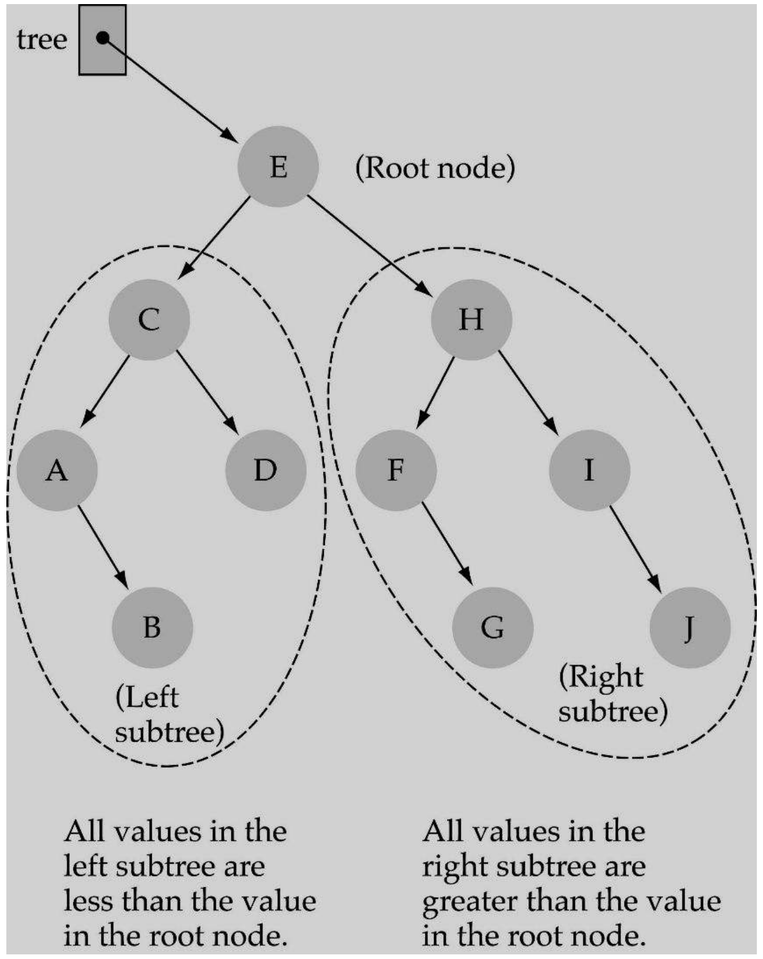
Insert − Inserts an element in a tree.

Pre-order Traversal − Traverses a tree in a pre-order manner.

In-order Traversal − Traverses a tree in an in-order manner.

Post-order Traversal − Traverses a tree in a post-order manner.

How to search a binary search tree?

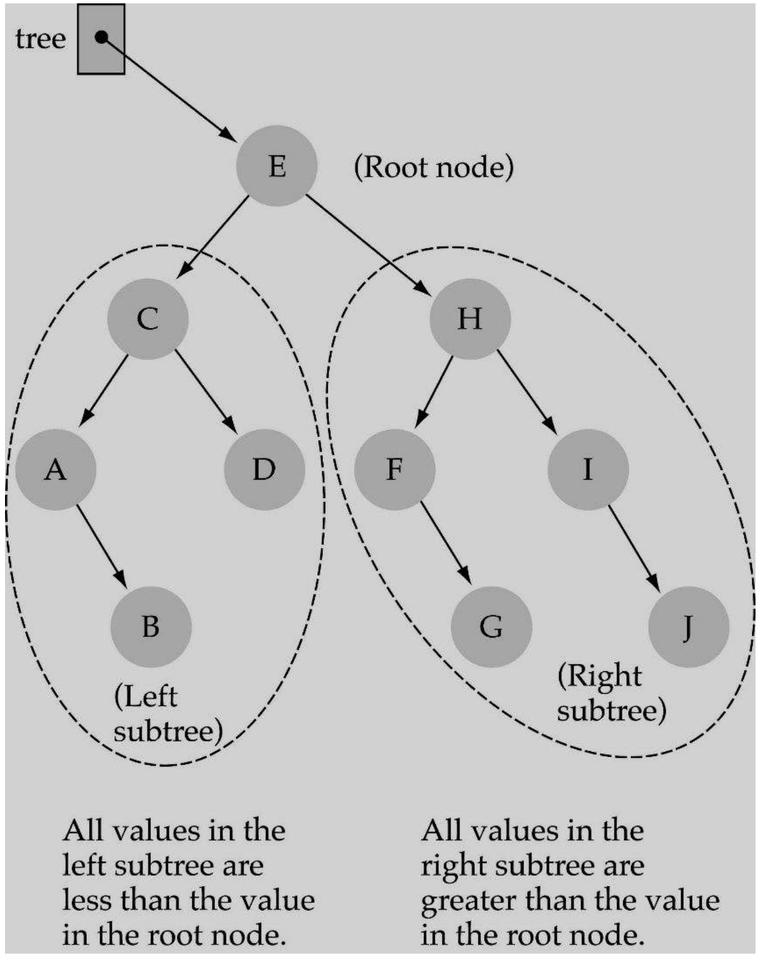


(1) Start at the root

(2) Compare the value of the item you are searching for with the value stored at the root

(3) If the values are equal, then *item found*; otherwise, if it is a leaf node, then *not found*

How to search a binary search tree?

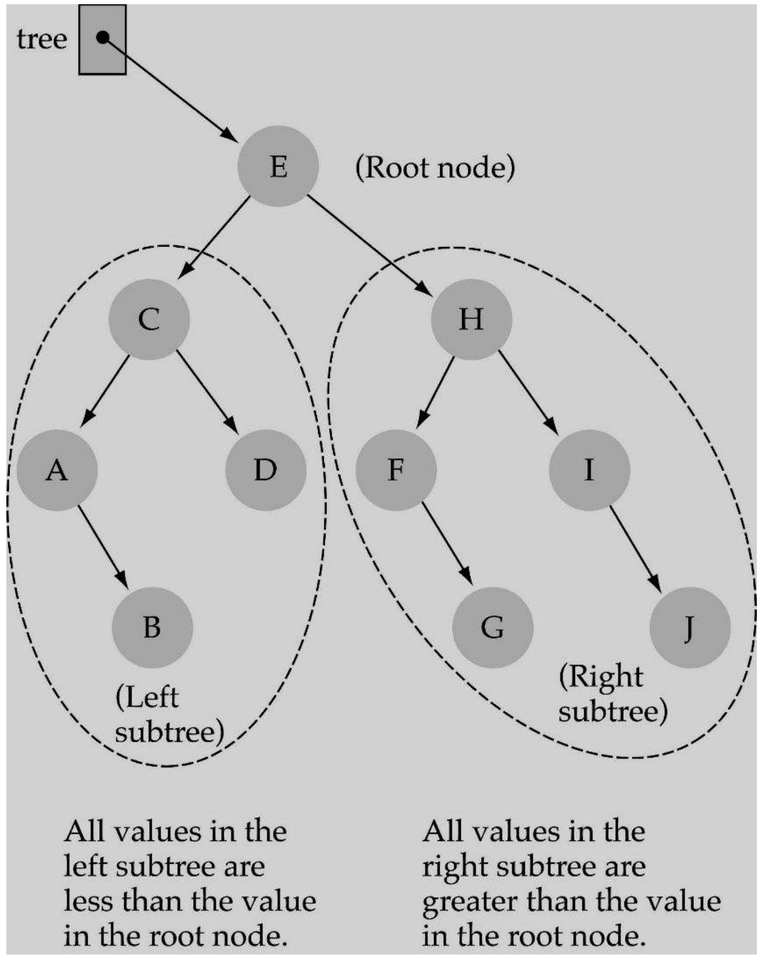


) If it is less than the value stored at the root, then search the left subtree

(5) If it is greater than the value stored at the root, then search the right subtree

(6) Repeat steps 2-6 for the root of the subtree chosen in the previous step 4 or 5

How to search a binary search tree?



Is this better than searching a linked list?

Yes !! ---> O(logN)

Search – Searches an element in a tree.

```
struct node {
   int data;
   struct node *leftChild;
   struct node *rightChild;
};
```

```
struct node* search(int data){
   struct node *current = root;
   printf("Visiting elements: ");

   while(current->data != data){

      if(current != NULL) {
         printf("%d ",current->data);

         //go to left tree
         if(current->data > data){
            current = current->leftChild;
         }  //else go to right tree
         else {
            current = current->rightChild;
         }

         //not found
         if(current == NULL){
            return NULL;
         }
      }
   }

   return current;
```

Insert Operation

- Whenever an element is to be inserted, first locate its proper location.
- Start searching from the root node, then if the data is less than the key value, search for the empty location in the left subtree and insert the data.
- Otherwise, search for the empty location in the right subtree and insert the data.

```
void insert(int data) {
   struct node *tempNode = (struct node*) malloc(sizeof(struct node));
   struct node *current;
   struct node *parent;

   tempNode->data = data;
   tempNode->leftChild = NULL;
   tempNode->rightChild = NULL;

   //if tree is empty
   if(root == NULL) {
      root = tempNode;
   } else {
      current = root;
      parent = NULL;

      while(1) {
         parent = current;

         //go to left of the tree
         if(data < parent->data) {
            current = current->leftChild;
            //insert to the left

            if(current == NULL) {
               parent->leftChild = tempNode;
               return;
            }
         } //go to right of the tree
         else {
            current = current->rightChild;

            //insert to the right
            if(current == NULL) {
               parent->rightChild = tempNode;
               return;
            }
         }
      }
   }
}
```

Inorder Traversal

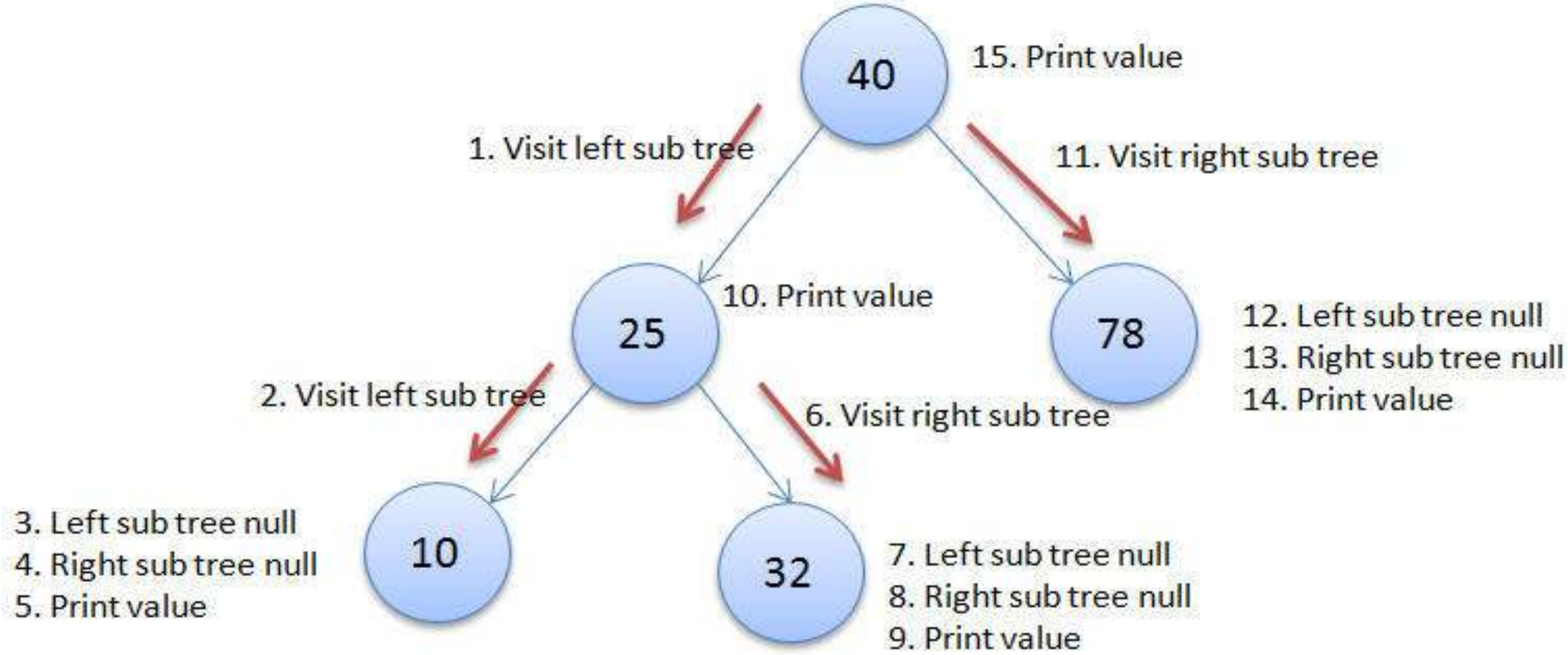
```
void inorder(Node root)
{
      if (root != null)
     {
         inorder(root->left);
         printf("%d", root->data);
         inorder(root->right);
      }
}
```

Preorder Traversal

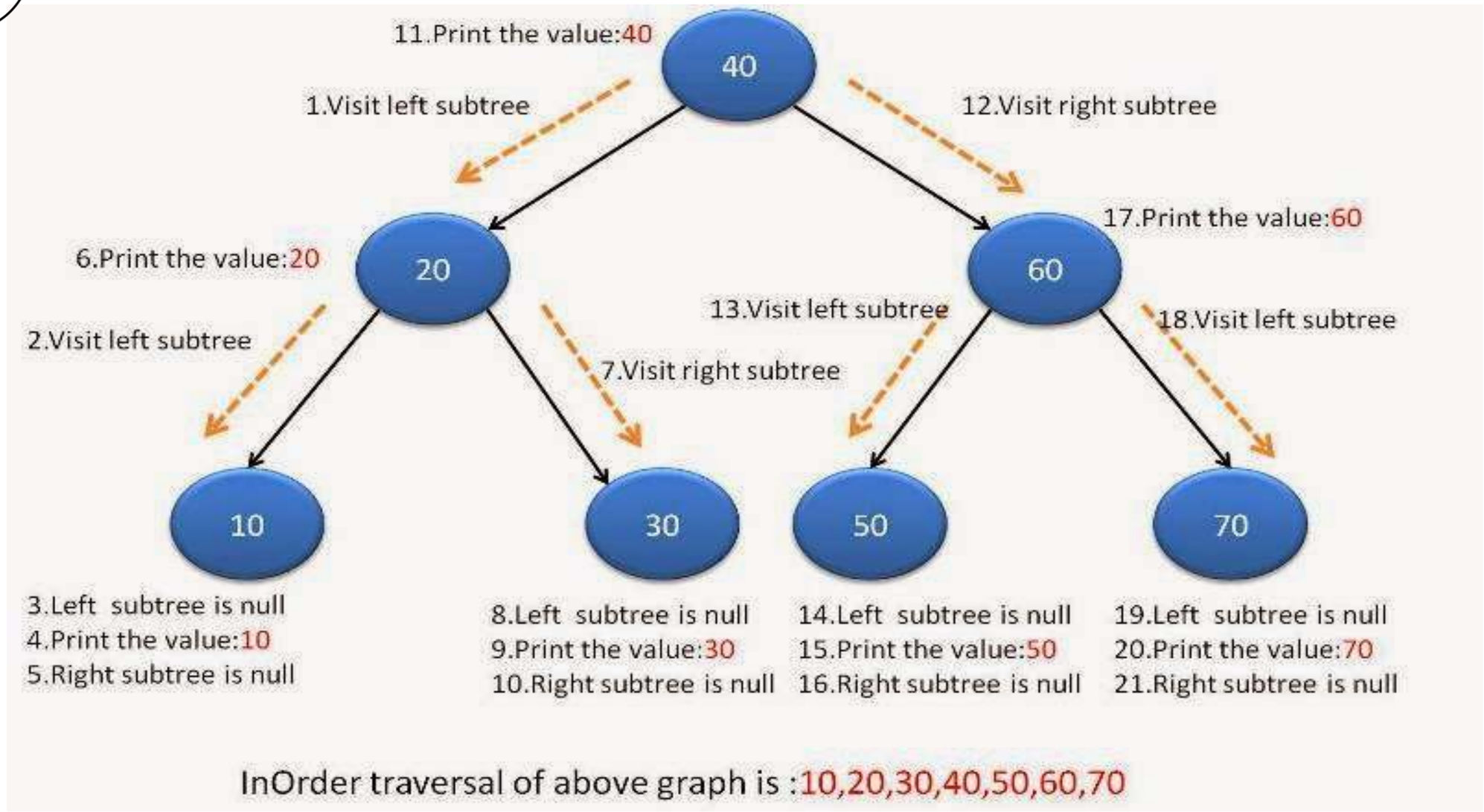
```
void preorder(Node root)
{
      if (root != null)
      {
          printf("%d", root->data);
         preorder(root->left);
         preorder(root->right);
      }
}
```

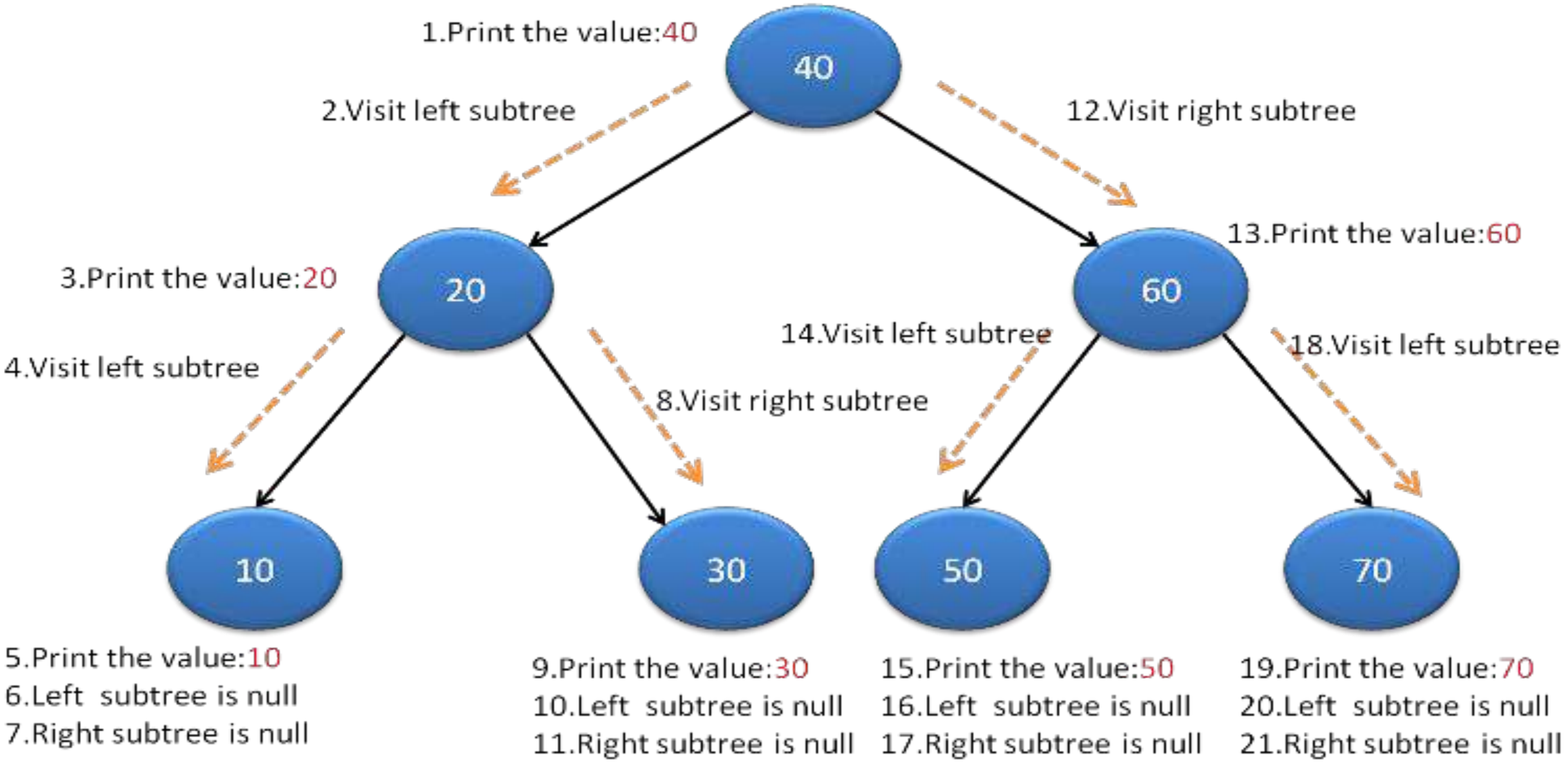
Postorder Traversal

```
void postorder(Node root)
{
      if (root != null) {
        postorder(root->left);
        postorder(root->right);
    printf("%d", root->data);
}
```



The above POSTORDER traversal gives: **10, 32, 25, 78, 40**


11.Print the value:40
40
1.Visit left subtree
12.Visit right subtree
17.Print the value:60
6.Print the value:20
20
60
13.Visit left subtree
18.Visit left subtree
2.Visit left subtree
7.Visit right subtree
10
30
50
70
3.Left subtree is null
4.Print the value:10
5.Right subtree is null
8.Left subtree is null
9.Print the value:30
10.Right subtree is null
14.Left subtree is null
15.Print the value:50
16.Right subtree is null
19.Left subtree is null
20.Print the value:70
21.Right subtree is null
InOrder traversal of above graph is :10,20,30,40,50,60,70

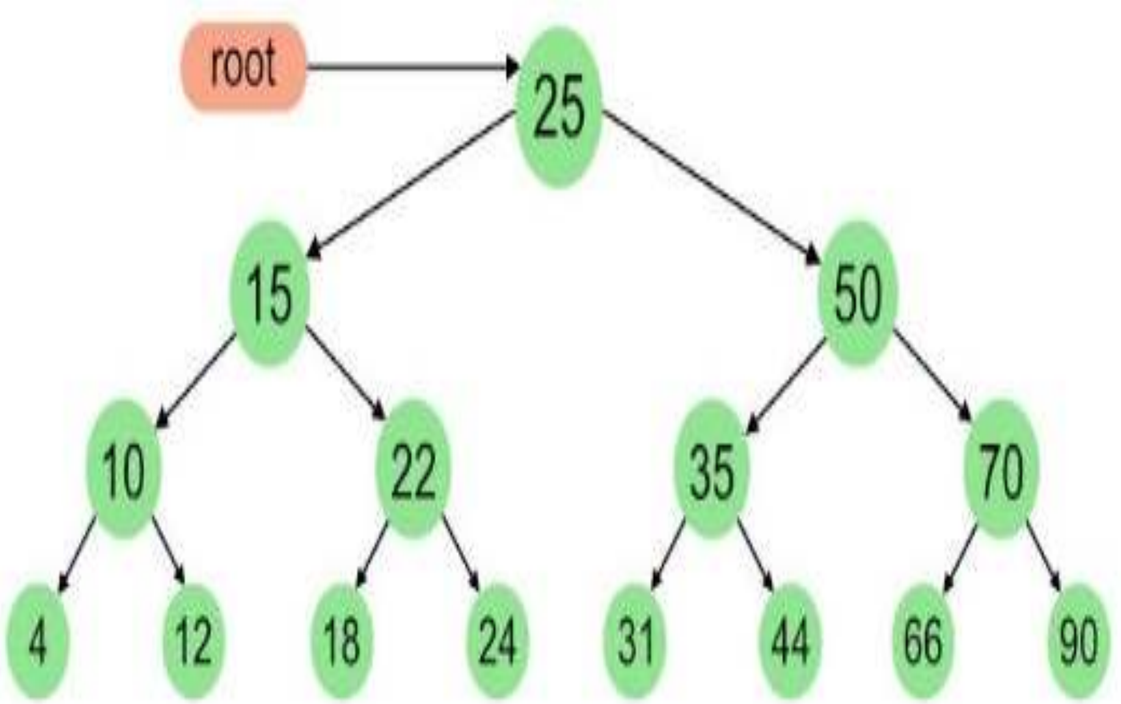


PreOrder traversal of above graph is :40,20,10,30,60,50,70

Application of Traversal Sorting a BST

- Observe the output of the inorder traversal of the BST example two slides earlier
- It is sorted
- As a general rule, if you output the keys (data) of the nodes of a BST using inorder traversal, the data comes out sorted in increasing order

Find the inorder , preorder, postorder for the following BST



InOrder(root) visits nodes in the following order:

4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

A Pre-order traversal visits nodes in the following order:

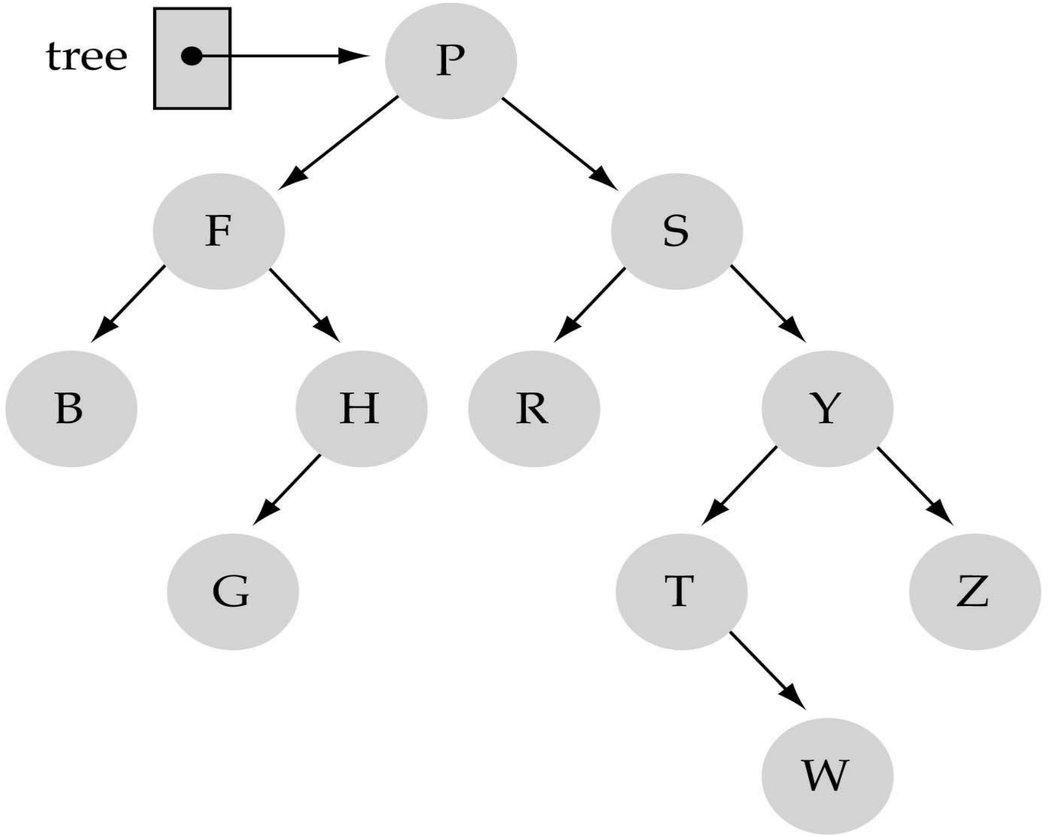
25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:

4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Tree Traversals: another example

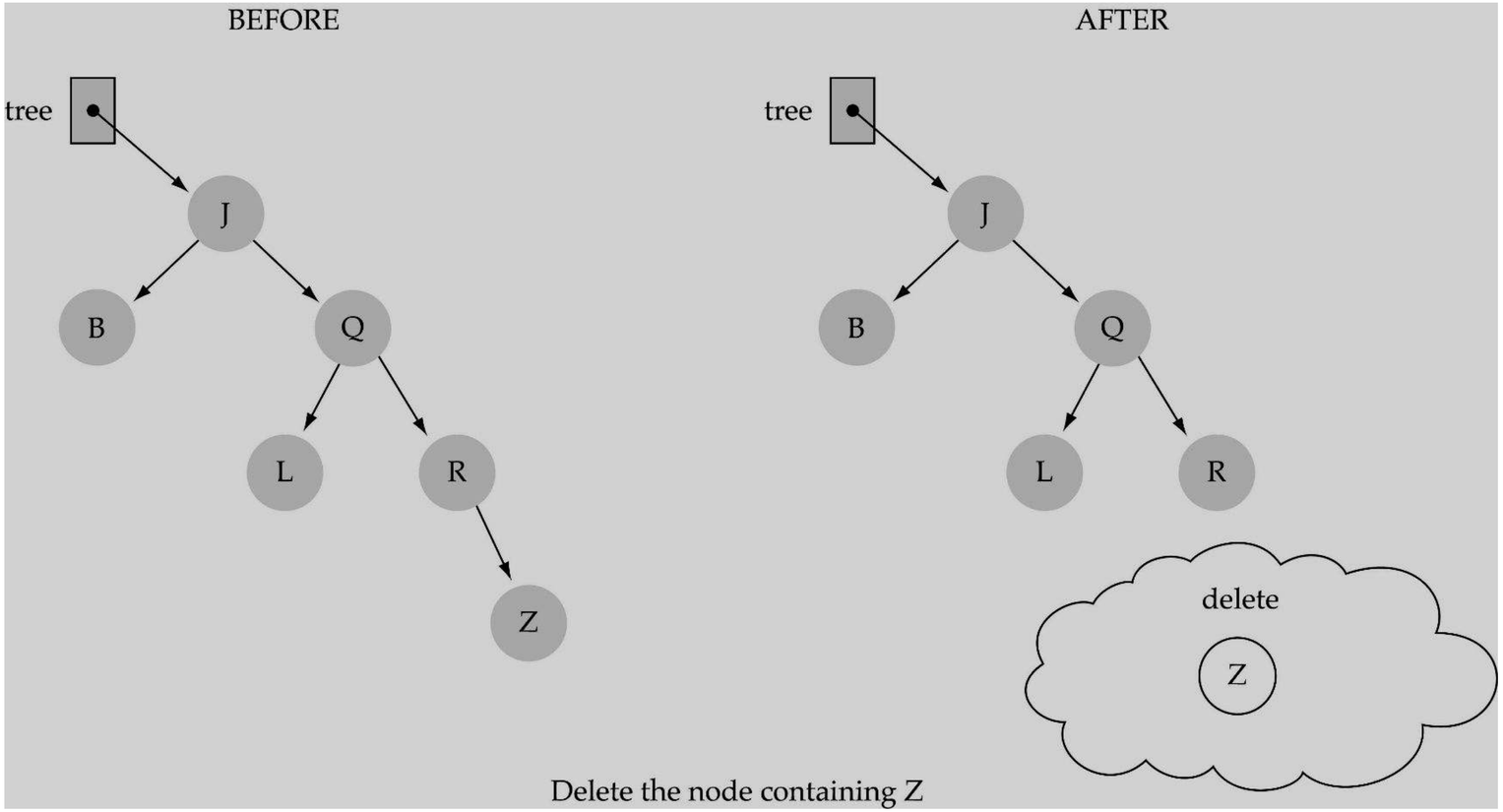


Inorder:	B	F	G	H	P	R	S	T	W	Y	Z
Preorder:	P	F	B	H	G	S	R	Y	T	W	Z
Postorder:	B	G	H	F	R	W	T	Z	Y	S	P

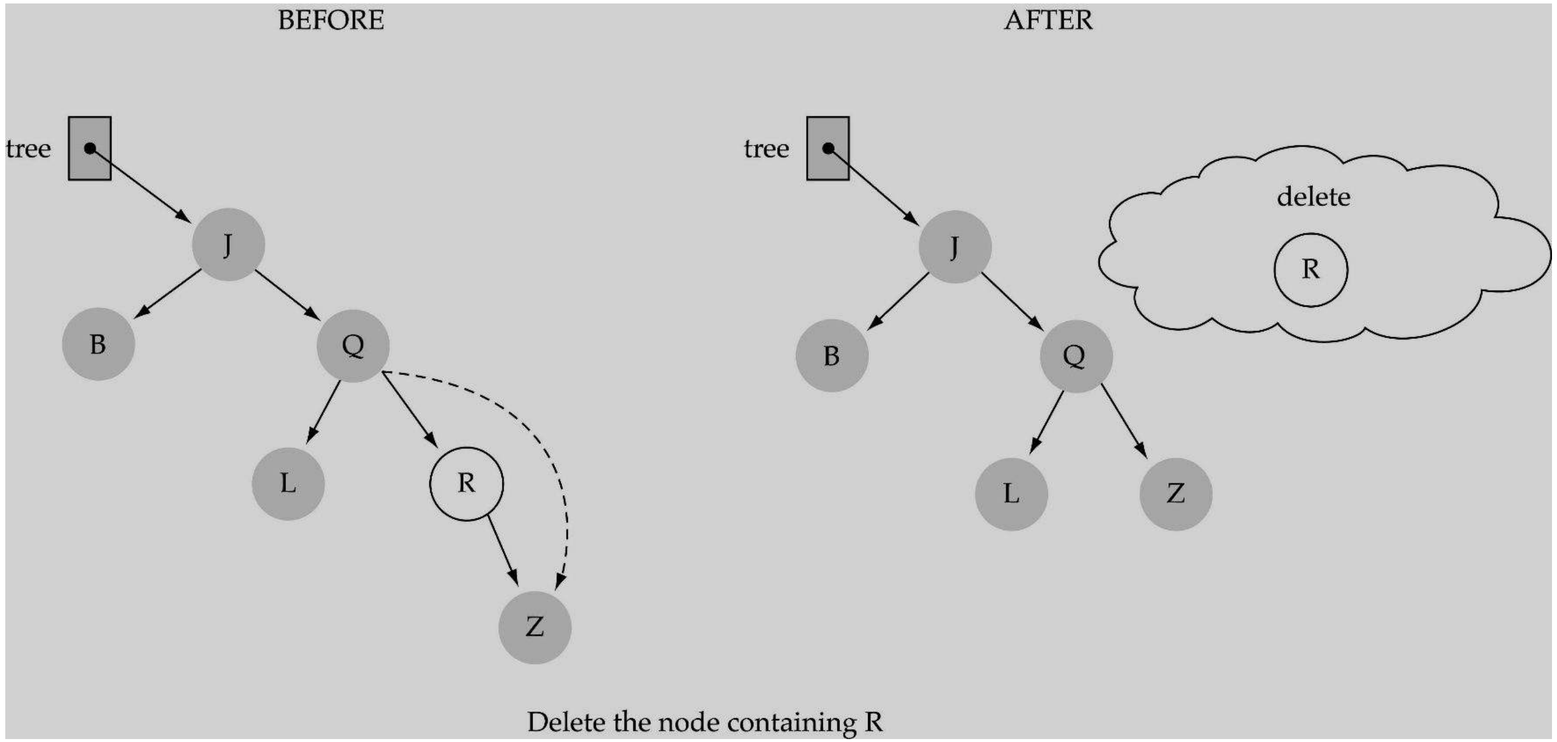
Delete Operation in BST

- First, find the item; then, delete it
- Binary search tree property must be preserved!!
- We need to consider three different cases:
  - **(1)** Deleting a leaf
  - **(2)** Deleting a node with only one child
  - **(3)** Deleting a node with two children

(1) Deleting a leaf



(2) Deleting a node with only one child



case 1 : No child

case 2 : one child

case 3 : 2 children

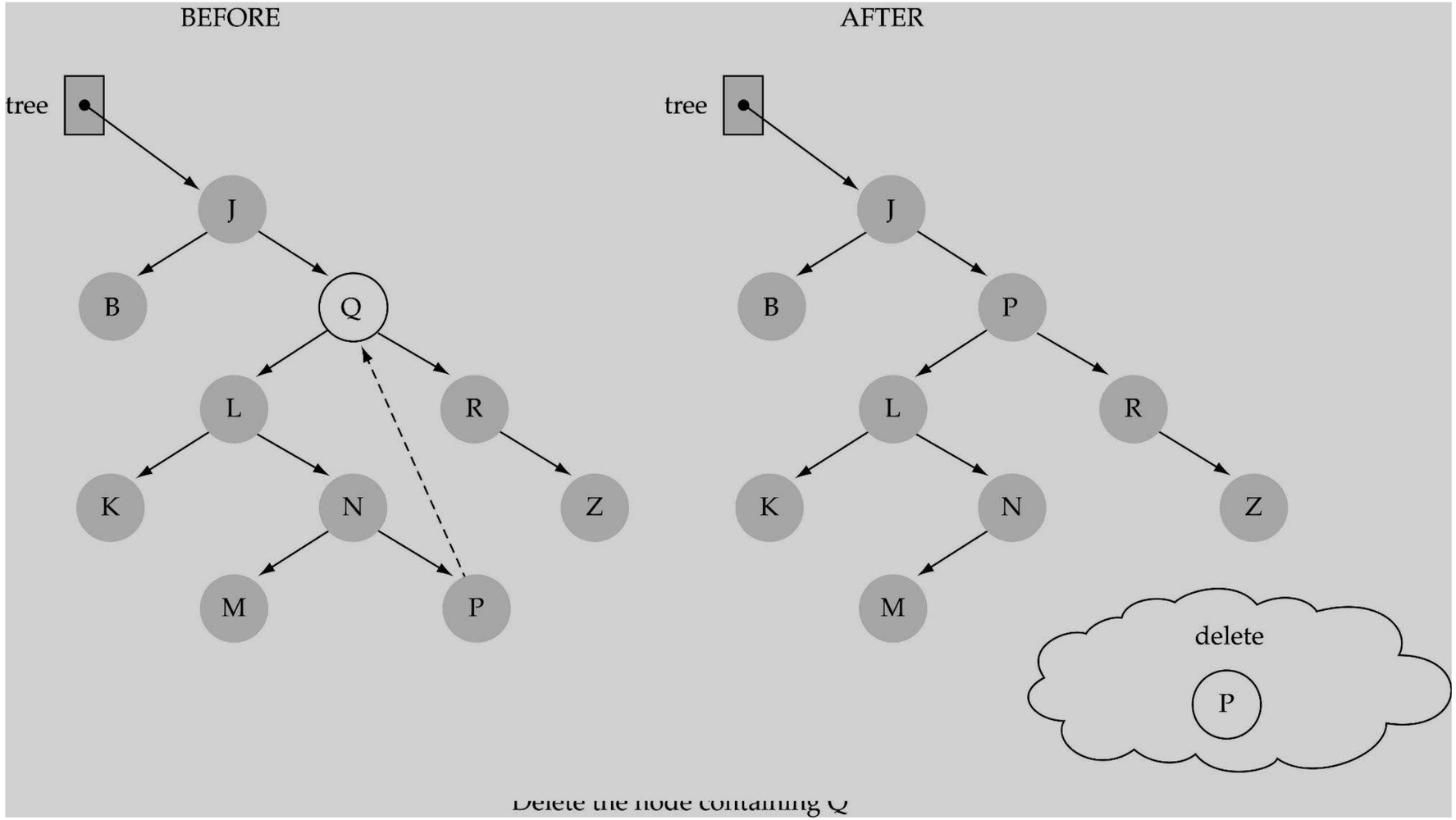
Find min in right
copy the value
in targetted node
Delete duplicate
from right-Subtree

Find max in left
copy the value
in targetted node
Delete duplicate
from left-Subtre

Inorder successor

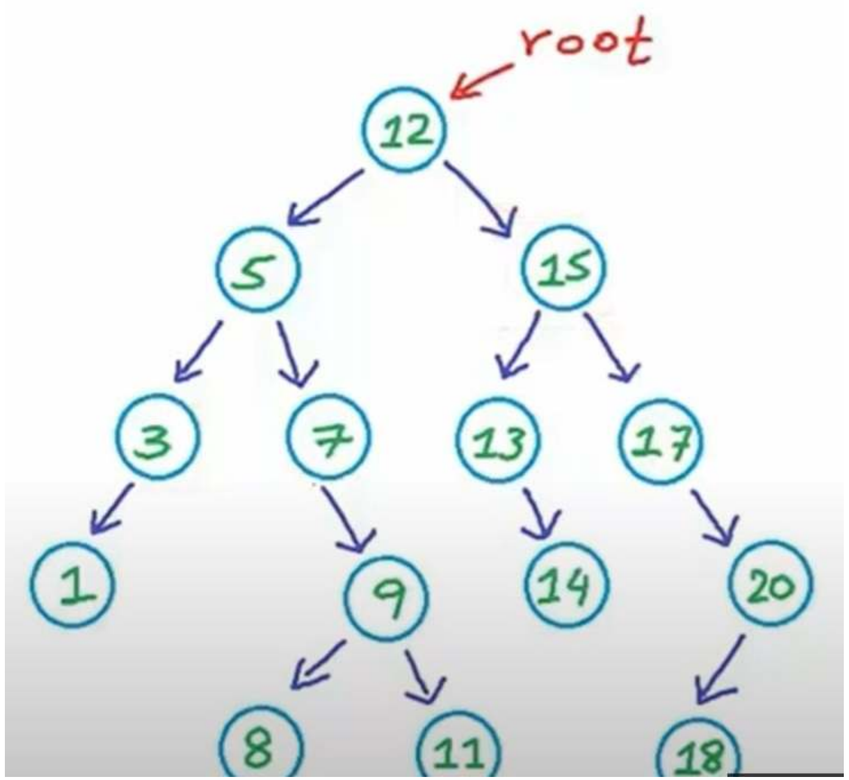
Inorder predecessor

(3) *Deleting a node with two children(find max in left subtree)*

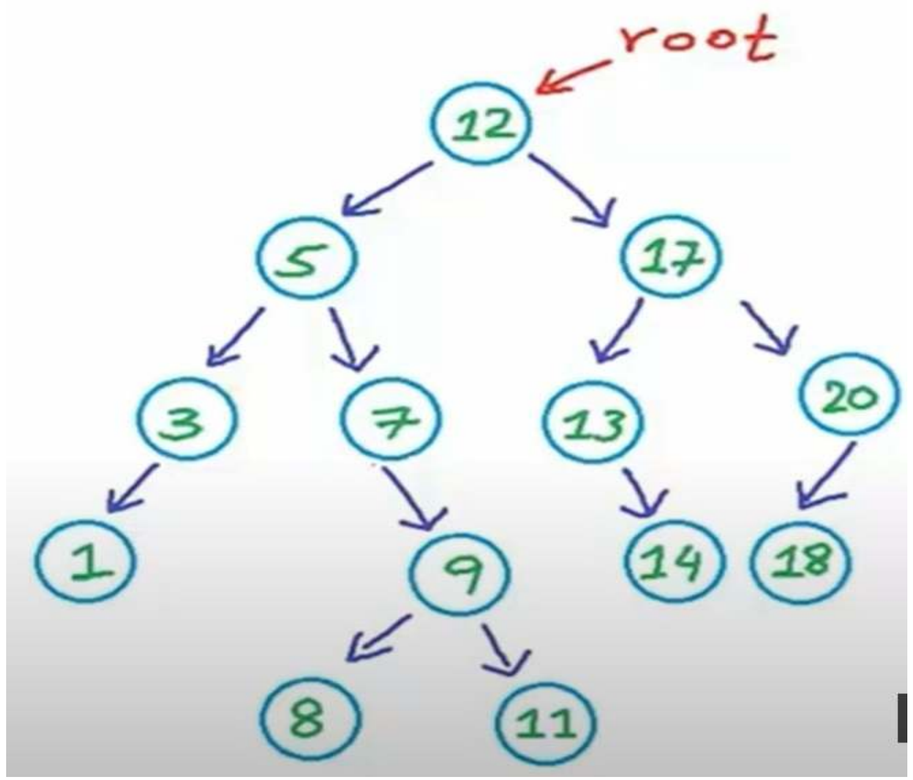


Delete the node containing Q

(3) *Deleting a node with two children(find min in right subtree)*



Before deleting 15



After deletion

- # Algorithm

1. Perform search for value X
2. If X is a leaf, delete X
3. Else // must delete internal node

   a) Replace with largest value Y on left subtree
   OR smallest value Z on right subtree

   b) Delete replacement value (Y or Z) from subtree

Try the implementation!!!!